Assignment 5: Optimising Attention on a CPU

CS6886: Systems for Deep Learning
Hayagriv Venkatesh ED23B024

Q1: Your Machine

My machine is a M5 Macbook pro.

The Core and cache values were obtained using `sysctl` command; memory and model details using `system_profiler`; clocks from the power management frequency tables. `powermetrics` under a ten-process CPU load supplied the loaded clocks. Its E cluster hardware reading was 2.964 GHz, whereas per core counters reported 3.048 GHz, the hardware cluster value is recorded above.

Each vector FMA performs $4 \times 2 = 8$ FP32 FLOPs.

$$
\begin{aligned}
P_{(performance)} &= 4.608 \times 4 \times 4 \times 2 = 147.456 \text{ GFLOP/s}, \\
P_{(efficiency)} &= 3.048 \times 3 \times 4 \times 2 = 73.152 \text{ GFLOP/s}, \\
P_{\text{allcores}} &= 4P_{(performance)} + 6P_{(efficiency)} = 1028.736 \text{ GFLOP/s}, \\
B &= 153 \text{ GB/s} \quad \text{(Apple's standard value)}, \\
I_{\text{ridge,1}} &= 147.456/153 = 0.964 \text{ FLOP/B}, \\
I_{\text{ridge,all}} &= 1028.736/153 = 6.724 \text{ FLOP/B}.
\end{aligned}
$$

These NEON ceilings assume maximum clocks and full utilization, they exclude matrix accelerators. All core maximum clocks need not be sustained simultaneously. The bandwidth is theoretical, not measured CPU bandwidth. The one-core envelope also uses full vendor bandwidth. The 4/3 pipeline model is based on published microbenchmarks.

Q2: Naive Baseline

The baseline was compiled with Apple Clang 21.0.0 using `-O3 -march=native` and `make naive`. Each input receives one warm-up and five timed runs, the median is result. The driver counts the two matrix multiplications as $F = 4N^2d$ FLOPs:

$$
\text{GFLOP/s} = \frac{4N^2d}{t_{\text{seconds}}\, 10^9}, \qquad \%\text{peak} = 100\frac{\text{GFLOP/s}}{147.456}.
$$

N	d	Time (ms)	GFLOP/s	Peak (%)
1024	64	56.781	4.73	3.21
2048	64	237.226	4.53	3.07
4096	64	994.832	4.32	2.93
4096	128	2555.397	3.36	2.28

Scaling with N. At $d = 64$, doubling N increases runtime by 4.178 and 4.194. This is close to the $4\times$ increase expected from $O(N^2d)$. The score matrix grows from 4 to 16 to 64 MiB, the P-cluster L2 is 16 MiB. Increased cache pressure is consistent with the slightly worse scaling, but cache misses were not profiled.

Scaling with d. At $N = 4096$, doubling d gives $2.569\times$ time for approximately $2\times$ arithmetic work. Input/output working sets double, and the output multiplication's stride through V increases from 256 to 512 bytes.

The score and output multiplications each perform approximately $2N^2d$ FLOPs. Both have accumulation dependencies, the naive output loop walks down columns of row major V. The full score matrix is written and revisited. Softmax performs maximum finding, exponentials, summation, and normalization over every row. Temporary allocation/free are timed. Softmax time contributes to runtime although its operations are excluded from the driver's FLOP count. Low peak utilization therefore does not mean the CPU was idle so an exact stage breakdown would require profiling.

Q3: Optimisations

(a) Loops. Transpose K so keys are contiguous for an $i \to k \to j$ score loop. Reorder SV to $i \to j \to k$, reading rows of V. Clang confirms four lane SIMD on both accumulation loops. Remaining costs include repeated accumulator array traffic and the full score matrix.

(b) Tiled. Used $32 \times 64 \times 64$ cache blocks and 4×8 register tiles. Unrolled inner loops retain 32 partial sums per microkernel. The three idealized blocks occupy $4(32 \cdot 64 + 64 \cdot 64 + 32 \cdot 64) = 32$ KiB. This improves reuse, while score-matrix materialization, softmax, and intermediate stores remain.

(c) Fused. Finish scores, softmax, and output for one query row, then reuse the scratch row. At $N = 4096$, score scratch falls from 64 MiB to 16 KiB. Dot-product dependencies and limited cross-query reuse remain, less storage alone does not guarantee the fastest kernel.

(d) Best. Combine transposed keys, 32 query blocked fusion, and explicit ARM NEON 4×16 microkernels. Sixteen vector registers hold 64 partial sums. The score block is 512 KiB at $N = 4096$. The 4×16 shape was selected among several tested on $N = 4096, d = 64$. Remaining costs include softmax, packing, and data movement.

N	d	Variant	Time (ms)	GFLOP/s	Peak (%)	Speedup
1024	64	loops	15.713	17.08	11.58	3.61×
1024	64	tiled	8.125	33.04	22.41	6.99×
1024	64	fused	17.142	15.66	10.62	3.31×
1024	64	best	7.010	38.29	25.97	8.10×
2048	64	loops	45.133	23.79	16.13	5.26×
2048	64	tiled	20.715	51.83	35.15	11.45×
2048	64	fused	64.125	16.74	11.35	3.70×
2048	64	best	16.458	65.24	44.24	14.41×
4096	64	loops	173.956	24.69	16.74	5.72×
4096	64	tiled	87.613	49.02	33.24	11.35×
4096	64	fused	262.914	16.34	11.08	3.78×
4096	64	best	71.555	60.02	40.70	13.90×
4096	128	loops	281.171	30.55	20.72	9.09×
4096	128	tiled	150.056	57.24	38.82	17.03×
4096	128	fused	555.187	15.47	10.49	4.60×
4096	128	best	117.218	73.28	49.70	21.80×

The best variant wins on all four current inputs, reaching 21.80× speedup at $N = 4096, d = 128$. All optimized variants pass the 10^{-4} normalized maximum error tolerance. The largest observed error is approximately 1.2×10^{-6}.

Q4: OpenMP

$$S_p = \frac{T_1}{T_p}, \qquad E_p = \frac{S_p}{p} \times 100\%.$$

The changes - Kept the best NEON microkernel and 32-query blocks. An OpenMP parallel region assigns blocks with `schedule(dynamic,1)`, so faster cores can take more tasks. Each worker owns a private score buffer. Shared inputs and transposed keys are read only, and output rows are disjoint. Transposition remains serial and the region completes before shared storage is freed. The Makefile enables Apple Clang's OpenMP frontend and links Homebrew `libomp`.

Median of five timed runs after one warm up. Dynamic team sizing and explicit thread binding were disabled. For each input, T_1 is the same OpenMP implementation at one thread:

N	d	Threads	Time (ms)	GFLOP/s	S_p	E_p (%)
1024	64	1	4.039	66.46	1.00	100.0
1024	64	2	2.299	116.76	1.76	87.8
1024	64	4	1.285	208.90	3.14	78.6
1024	64	6	1.104	243.15	3.66	61.0
1024	64	8	1.008	266.31	4.01	50.1
1024	64	10	0.978	274.47	4.13	41.3
2048	64	1	16.451	65.27	1.00	100.0
2048	64	2	8.869	121.07	1.85	92.7
2048	64	4	5.471	196.26	3.01	75.2
2048	64	6	4.114	261.00	4.00	66.6
2048	64	8	3.582	299.76	4.59	57.4
2048	64	10	3.313	324.10	4.97	49.7
4096	64	1	70.962	60.52	1.00	100.0
4096	64	2	37.859	113.45	1.87	93.7
4096	64	4	20.731	207.18	3.42	85.6
4096	64	6	17.641	243.47	4.02	67.0
4096	64	8	15.336	280.06	4.63	57.8
4096	64	10	13.288	323.22	5.34	53.4
4096	128	1	119.769	71.72	1.00	100.0
4096	128	2	63.096	136.14	1.90	94.9
4096	128	4	34.641	247.97	3.46	86.4
4096	128	6	27.809	308.89	4.31	71.8
4096	128	8	24.005	357.84	4.99	62.4
4096	128	10	26.848	319.95	4.46	44.6

At $N = 4096, d = 64$, ten threads give 5.34× speedup. For $d = 128$, eight threads are fastest (4.99×), ten threads are slower. Efficiency generally decreases with thread count.

Q5: Roofline

Q1 values,

$$P(I) = \min(P_{\text{peak}}, 153I), \qquad I = F/D, \qquad F = 4N^2d.$$

Model. Reading Q, K, V and writing O costs $16Nd$ modeled bytes. A full score matrix adds $16N^2$ bytes: score write, softmax read/write, and output stage read. The key transpose adds an $8Nd$ write/read charge that cache may absorb. Row/block scratch and repeated accesses are assumed cached:

$$\begin{aligned} D_{\text{naive}} &= 16Nd + 16N^2, \\ D_{\text{loops}} = D_{\text{tiled}} &= 24Nd + 16N^2, \\ D_{\text{fused}} &= 16Nd, \\ D_{\text{best}} = D_{\text{omp}} &= 24Nd. \end{aligned}$$

This analytical DRAM streaming estimate omits write allocation, extra spills, and cluster duplication; it is not measured traffic. Softmax time is included in performance, while its operations are excluded from the FLOP count.

Variant	Threads	I (FLOP/B)	GFLOP/s
naive	1	15.754	4.32
loops	1	15.634	24.69
tiled	1	15.634	49.02
fused	1	1024.000	16.34
best	1	682.667	60.02
omp	10	682.667	323.22

Interpretation. All points use $N = 4096, d = 64$ and lie to the right of their applicable idealized ridge. This does not prove their actual bottlenecks. Loops improves SIMD execution and tiling improves cache/register reuse despite similar modeled DRAM intensity. Fused removes the large score matrix but retains dot product dependencies. Best combines NEON and blocked fusion, OpenMP raises throughput.

